

中国科学技术大学

2019~2020 学年第 2 学期考试试卷 (A 卷)

考试科目: 微机原理与嵌入式系统

班级: _____

姓名: _____

学号: _____

成绩: _____

一、填空题 (共 20 分)

- 冯·诺依曼从逻辑结构上将计算机划分为____、____、____、输入设备和输出设备五大部件。
- 在字长 $n=8$ 的计算机中, 考虑补码表示法, 一个有符号整数 $1000\ 0000b$ 表示的十进制真值是____, 一个无符号整数 $1111\ 1111b$ 表示的十进制真值是____。
- ____、____和控制相关冲突会导致指令流水线出现断流。
- 常见存储器类型有____、____和光存储器等。
- 主存块到 Cache 块的地址映射包括____、____、____三种地址映射方案。
- Cortex-M3 处理器进入异常处理子程序之前需要把____、____、LR、R0~R3、R12 等寄存器的值保护起来。
- 常见的 I/O 端口编址方式有____、____两种。
- 一个 8 位有符号数 “ $1000\ 0001b$ ” 经符号扩展转换至 16 位有符号数, 结果为____。
- 总线操作周期一般可以分为: 请求及仲裁、____、____及结束等阶段。
- Cortex-M3/M4 中断向量表中保存的是____。

二、单项选择题 (共 10 分)

- 以下属于串行接口的是 ()。
A. I²C B. PCI C. AHB D. APB
- Cortex-M3 内部的 NVIC 支持 () 个外部中断的管理。
A. 127 B. 128 C. 240 D. 255
- Cortex-M3 采用的是以下哪种堆栈 ()。
A. Full Ascending Stack C. Empty Ascending Stack
B. Full Descending Stack D. Empty Descending Stack
- Cortex-M3 处理器中寄存器 R14 代表了 ()。
A. 通用寄存器 C. 程序计数器
B. 链接寄存器 D. 程序状态寄存器
- 以下关于 Cortex-M3 描述正确的有 ()。
A. 存储器访问指令只能支持对齐数据传输
B. 特定的存储器区域可支持位段 (Bit-Band, 也称作位带) 操作
C. 存放在 SRAM (0x2000 0000 — 0x3FFF FFFF) 区的代码是不允许执行的
D. Cortex-M3 不会改变代码的执行顺序, 因此不需要存储器屏障指令
- 以下关于 T32 指令集寻址方式描述不正确的是 ()。
A. 前变址寻址时, 当前操作数地址由基址与偏移加和形成
B. 后变址寻址时, 当前操作数地址即基址寄存器中的数值
C. 寄存器间接寻址时, 把基址寄存器的值与地址偏移量相加得到操作数地址
D. 立即数寻址时, 操作数本身包含在指令中
- 以下关于微型计算机存储器系统描述不正确的是 ()。
A. 内存即主存储器 C. 闪存属于 ROM
B. 缓存的访问速度快于闪存 D. 内存属于 ROM

18. 以下关于微机和嵌入式系统说法不正确的是 ()。
- A. 嵌入式系统的内核小
 - B. 嵌入式系统的专用性强
 - C. 嵌入式系统针对多任务执行优化
 - D. 在通用任务处理方面微机比嵌入式系统有优势
19. 以下关于指令周期说法正确的是 ()。
- A. 指令周期就是 T 周期 (时钟周期)
 - B. 总线周期就是 T 周期 (时钟周期)
 - C. 时钟频率越高, 时钟周期就越长
 - D. CPU 周期也称为总线周期
20. 以下关于 AHB 说法正确的是 ()。
- A. AHB Lite 支持多主控设备
 - B. AHB 的突发传输不仅支持单个数据的传输, 也支持数据块的传输
 - C. AHB 的突发传输在传输开始前必须指定传输数据的长度
 - D. AHB 所传输的数据本身和数据的地址需要在同一个时钟周期推送到总线上

三、判断题 (共 10 分)

- 21. Cortex-M3 处理器内核采用了冯诺依曼结构的三级流水线. ()
- 22. Cortex-M3 处理器工作在线程模式下时, 代码一定是非特权的. ()
- 23. Cortex-M3 中某些特殊寄存器只有特权访问等级才能访问. ()
- 24. 在单主控设备、多从控设备的情形下是不需要总线仲裁的. ()
- 25. AHB 所传输的数据及其地址必须在同一个时钟周期推送到总线上. ()
- 26. 冯·诺依曼结构和哈佛结构是两种不同的 ISA. ()
- 27. Cortex-M3 系列处理器支持 Thumb-2 指令集. ()
- 28. Cortex-M3 中 MOV 指令可将数据从一个寄存器送到寄存器的特定位置. ()
- 29. Cortex-M3 中异常的类型号越小, 该异常所对应的优先级就越高. ()
- 30. 存储器字扩展时, 各芯片不同位数据线分别于数据总线 D₇~D₀ 位并联. ()

四、简答题 (共 24 分)

- 31. 作为一种性能指标, MIPS 是否能客观反映计算机的运算速度? 为什么?
- 32. 简述机器指令、微程序、微指令的联系.
- 33. 简要对比 SRAM 和 DRAM.
- 34. 为什么 AMBA 总线中没有定义电气特性和机械特性?
- 35. 接口电路的输入需要用缓冲器, 而输出需要用锁存器. 为什么?
- 36. 简述 Cortex-M 处理器的中断优先级分组机制.

五、程序设计题 (共 16 分)

37. 阅读以下代码并回答问题 (提示: 汇编指令可参考附录 1)。

```
MOV R0, #76
MOV R0, #-1
CMP R0, R1
MOVH1 R2, #100    ;①
MOVLS R2, #50      ;②
LOOP B LOOP
```

- (1) 这段代码的功能是?
 - (2) 执行这段代码后 R2 寄存器里完整内容是什么? (注意 R2 位数, 写出 16 进制数值)
 - (3) 如果代码①和代码②分别改为 MOVGT R2, #100 和 MOVLE R2, #50. 那么执行完这段代码后 R2 寄存器里完整内容是什么? (写出 16 进制数值)
38. 汇编程序中调用 C 函数实现求两个整数相加的和 (汇编程序和 C 函数要有完整代码)。

六、综合设计题（共 20 分）

- 在一个 STM32 点亮 LED 的应用中，部分程序代码如附录 2 所示。请回答以下问题。
- 简述 GPIO_Configuration 函数对 IO 接口配置的步骤。
 - 分析该程序，LED 分别连接在哪些 IO 引脚上？请画出一个简明硬件连线图；根据此连线图，当引脚输出高电平时，是点亮还是熄灭 LED？
 - 分析循环点亮 LED 代码，补充相应的注释①②。
 - 可以用定时器实现精确循环（点亮）功能。已知系统时钟为 72MHz，采用定时器 TIM2 产生周期为 500ms 的定时时间间隔（控制 LED 的亮灭）。请简述定时器配置的主要步骤，并补充完整定时函数的空余部分③④。

附录 1：ARM 汇编指令相关信息

CMP 指令

寄存器内数值与立即数比较的汇编语法为“CMP[cond] [q]<Rd>, <Ra>”，该指令将更新标志位 APSR.N、APSR.Z、APSR.C、APSR.V。CMP 指令类似 SUB 指令（如“SUB R2, R1”指令 R2-R1 的结果存入 R2），只不过 CMP 指令仅更新标识位，但不写目的寄存器。

B 指令

跳转指令，其典型汇编语法为“B[cond] <label>”。

T32 指令集的条件码含义

表 1 整数运算时 T32 指令集的条件码含义（仅含部分条件码）

条件码	助记符	含义	APSR 标志位取值
0000	EQ	等于	Z == 1
0010	CS	产生了进位	C == 1
0100	MI	负数	N == 1
0110	VS	溢出	V == 1
1000	HI	无符号比较大于	C == 1 且 Z == 0
1001	LS	无符号比较小于等于	C == 0 或 Z == 1
1010	GE	有符号比较大于等于	N == V
1011	LT	有符号比较小于	N != V
1100	GT	有符号比较大于	Z == 0 且 N == V
1101	LE	有符号比较小于等于	Z == 1 或 N != V

附录 2：综合设计题程序代码

```
void GPIO_Configuration(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2 | GPIO_Pin_3;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
}
//...
while (1)
{
    /*循环点亮LED*/
    GPIO_WriteBit(GPIOA, GPIO_Pin2, (BitAction)0x01); //①
    Delay(0xFFFFF);
    GPIO_WriteBit(GPIOA, GPIO_Pin2, (BitAction)0x00); //②
    Delay(0xFFFFF);
    GPIO_WriteBit(GPIOA, GPIO_Pin3, (BitAction)0x01);
    Delay(0xFFFFF);
    GPIO_WriteBit(GPIOA, GPIO_Pin3, (BitAction)0x00);
    Delay(0xFFFFF);
}
//...
void TIM2_Delay500MS()
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    TIM_TimeBaseStructure.TIM_Prescaler = ③;
    TIM_TimeBaseStructure.TIM_Period = ④;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);
    TIM_ClearFlag(TIM2, TIM_FLAG_Update);
    TIM_Cmd(TIM2, ENABLE);
    while (TIM_GetFlagStatus(TIM2, TIM_FLAG_Update) == RESET);
}
```